

Глава 4: Основы управления меканум-приводом и лидаром

Учимся управлять роботом и чувствовать стены

В этой главе ты научишься основам управления меканум-роботом и использования лидара для обнаружения препятствий.

Что ты изучишь

1. Создание меканум-привода
 2. Движение вперед, вбок и поворот
 3. Чтение позиции робота (одометрия)
 4. Остановка после пройденной дистанции
 5. Использование лидара для остановки перед стеной
-

Зачем нужен Robot()?

Ты можешь спросить: зачем создавать `Robot()` и добавлять к нему компоненты? Почему бы не управлять моторами напрямую?

Robot управляет всем:

```
robot = Robot()
robot.drive = MecanumDrive(fl=3, fr=4, bl=1, br=2)
robot.lidar = RPLidarC1()
robot.start() # ← Запускает ВСЕ компоненты вместе!
```

Преимущества: - **Один старт/стоп** - Все компоненты запускаются и останавливаются вместе - **Общие соединения** - Компоненты могут общаться друг с другом (привод может использовать данные лидара) - **Автоматическая очистка** - Когда останавливаешь робота, всё останавливается корректно - **Проще код** - Один объект вместо множества отдельных частей

Без Robot (сложный способ):

```
# Пришлось бы управлять каждым мотором отдельно
motor1 = Servo42D(1)
motor2 = Servo42D(2)
motor3 = Servo42D(3)
motor4 = Servo42D(4)
lidar = RPLidarC1()

# Запустить каждый по отдельности
motor1.start()
motor2.start()
motor3.start()
motor4.start()
lidar.start()
```

```
# Рассчитывать, что должен делать каждый мотор...
# Это сложно!
```

Объект `Robot()` делает робототехнику простой - он берет на себя сложные вещи, чтобы ты мог сосредоточиться на классных штуках!

Создание меканум-привода

У твоего робота 4 колеса, которые могут двигаться в любом направлении. Давай настроим.

Конфигурация моторов

Сначала нужно узнать, какой мотор где находится. Моторы обозначены FL (передний-левый), FR (передний-правый), BL (задний-левый), BR (задний-правый).

Узнай ID своих моторов:

```
from evabot import Robot, MecanumDrive

robot = Robot()
robot.drive = MecanumDrive(fl=3, fr=4, bl=1, br=2, pattern='X')
robot.start()
```

Цифры (3, 4, 1, 2) - это CAN ID моторов. У твоего робота они могут отличаться - проверь метки на моторах.

Базовые движения

Теперь заставим робота двигаться!

Вперед и назад

```
import time
from evabot import Robot, MecanumDrive

robot = Robot()
robot.drive = MecanumDrive(fl=3, fr=4, bl=1, br=2, pattern='X')
robot.start()

# Движение вперед
robot.drive.move(vx=0.2) # 0.2 м/с вперед
time.sleep(2) # Движемся 2 секунды

# Стоп
robot.drive.halt()

robot.stop()
```

Что это делает: - $v_x=0.2$ означает движение вперед со скоростью 0.2 метра в секунду - $v_x=-0.2$ будет движение назад - `time.sleep(2)` ждет 2 секунды, пока робот движется - `halt()` останавливает робота

Движение влево и вправо

Меканум-колеса могут двигаться вбок без поворота!

```
import time
from evabot import Robot, MecanumDrive

robot = Robot()
robot.drive = MecanumDrive(fl=3, fr=4, bl=1, br=2, pattern='X')
robot.start()

# Движение влево
robot.drive.move(vy=0.2) # 0.2 м/с влево
time.sleep(2)

robot.drive.halt()

robot.stop()
```

Что это делает: - $v_y=0.2$ означает движение влево со скоростью 0.2 м/с - $v_y=-0.2$ будет движение вправо

Поворот на месте

```
import time
from evabot import Robot, MecanumDrive

robot = Robot()
robot.drive = MecanumDrive(fl=3, fr=4, bl=1, br=2, pattern='X')
robot.start()

# Поворот против часовой стрелки
robot.drive.move(vtheta=0.5) # 0.5 рад/с
time.sleep(2)

robot.drive.halt()

robot.stop()
```

Что это делает: - $v_{\theta}=0.5$ означает поворот против часовой стрелки со скоростью 0.5 радиан в секунду - $v_{\theta}=-0.5$ будет поворот по часовой стрелке

Упражнение 4.1: Движение во всех направлениях

Заставь робота: 1. Двигаться вперед 2 секунды 2. Двигаться вправо 2 секунды 3.

Двигаться назад 2 секунды 4. Двигаться влево 2 секунды

Решение

```
import time
from evabot import Robot, MecanumDrive

robot = Robot()
robot.drive = MecanumDrive(fl=3, fr=4, bl=1, br=2, pattern='X')
robot.start()

robot.drive.move(vx=0.2)
time.sleep(2)
robot.drive.halt()
time.sleep(0.5)

robot.drive.move(vy=-0.2)
time.sleep(2)
robot.drive.halt()
time.sleep(0.5)

robot.drive.move(vx=-0.2)
time.sleep(2)
robot.drive.halt()
time.sleep(0.5)

robot.drive.move(vy=0.2)
time.sleep(2)
robot.drive.halt()

robot.stop()
```

Движение на время

Вместо использования `time.sleep()`, можно указать роботу двигаться определенное время.

Использование `move_for()`

```
from evabot import Robot, MecanumDrive

robot = Robot()
robot.drive = MecanumDrive(fl=3, fr=4, bl=1, br=2, pattern='X')
robot.start()

# Движение вперед 3 секунды
robot.drive.move_for(3.0, vx=0.2)

# Движение влево 2 секунды
robot.drive.move_for(2.0, vy=0.2)

# Поворот 1.5 секунды
robot.drive.move_for(1.5, vtheta=0.5)
```

```
robot.stop()
```

Что это делает: - `move_for(3.0, vx=0.2)` двигает вперед со скоростью 0.2 м/с в течение 3 секунд, затем автоматически останавливается - Не нужно вызывать `halt()` - робот остановится сам - Проще, чем использовать `time.sleep()`!

Упражнение 4.2: Квадрат с `move_for()`

Заставь робота ехать по квадрату используя `move_for()`: - Каждая сторона: 2 секунды вперед - Каждый поворот: 1.6 секунды вращения

Решение

```
from evabot import Robot, MecanumDrive

robot = Robot()
robot.drive = MecanumDrive(fl=3, fr=4, bl=1, br=2, pattern='X')
robot.start()

for i in range(4):
    robot.drive.move_for(2.0, vx=0.2)
    robot.drive.move_for(1.6, vtheta=0.5)

robot.stop()
```

Чтение одометрии

Робот отслеживает свое положение, подсчитывая обороты колес. Это называется **одометрия**.

Проверка позиции робота

```
import time
from evabot import Robot, MecanumDrive

robot = Robot()
robot.drive = MecanumDrive(fl=3, fr=4, bl=1, br=2, pattern='X')
robot.start()

time.sleep(1) # Дадим одометрии инициализироваться

# Начинаем движение вперед
robot.drive.move(vx=0.2)

# Наблюдаем обновление позиции
for i in range(20):
    x = robot.odom.pose.x
    print(f"Позиция: {x:.2f} м")
    time.sleep(0.2)
```

```
robot.drive.halt()
robot.stop()
```

Что это показывает: - `robot.odom.pose.x` - позиция вперед/назад в метрах -
`robot.odom.pose.y` - позиция влево/вправо - `robot.odom.pose.theta` - угол поворота

Движение на дистанцию

Вместо времени или написания циклов, можно указать роботу точную дистанцию движения.

Использование `move_by()`

```
from evabot import Robot, MecanumDrive

robot = Robot()
robot.drive = MecanumDrive(fl=3, fr=4, bl=1, br=2, pattern='X')
robot.start()

# Движение точно на 1 метр вперед
robot.drive.move_by(dx=1.0)

# Движение назад на 0.5 метра
robot.drive.move_by(dx=-0.5)

# Движение влево на 0.3 метра
robot.drive.move_by(dy=0.3)

# Поворот на 90 градусов (1.57 радиан)
robot.drive.move_by(dtheta=1.57)

robot.stop()
```

Что это делает: - `move_by(dx=1.0)` двигает точно на 1 метр вперед, затем останавливается - Использует одометрию, чтобы знать когда остановиться - Намного точнее, чем движение по времени!

Упражнение 4.3: Квадрат с `move_by()`

Заставь робота ехать по квадрату $0.5\text{м} \times 0.5\text{м}$ используя `move_by()` : - Каждая сторона: 0.5 метра - Каждый поворот: 1.57 радиан (90 градусов)

Решение

```
from evabot import Robot, MecanumDrive

robot = Robot()
robot.drive = MecanumDrive(fl=3, fr=4, bl=1, br=2, pattern='X')
```

```
robot.start()

for i in range(4):
    robot.drive.move_by(dx=0.5)
    robot.drive.move_by(dtheta=1.57)

robot.stop()
```

Остановка по дистанции (ручной цикл)

Иногда `move_by()` недостаточно - нужно проверять датчики во время движения. Давай научимся писать свой цикл.

Простой цикл по дистанции

```
import time
from evabot import Robot, MecanumDrive

robot = Robot()
robot.drive = MecanumDrive(fl=3, fr=4, bl=1, br=2, pattern='X')
robot.start()

time.sleep(1)

# Запоминаем начальную позицию
start_x = robot.odom.pose.x

# Начинаем движение
robot.drive.move(vx=0.2)

# Продолжаем проверять, пока не проедем 1 метр
while True:
    current_x = robot.odom.pose.x
    distance = current_x - start_x

    if distance >= 1.0:
        break

    time.sleep(0.01)

# Стоп
robot.drive.halt()

print(f"Проехали {distance:.2f} метров")
robot.stop()
```

Как это работает: 1. Запоминаем, где начали (`start_x`) 2. Начинаем движение вперед 3. Продолжаем проверять текущую позицию в цикле 4. Когда проехали 1 метр, выходим из цикла 5. Останавливаем робота

Зачем писать свой цикл? - Когда нужно проверять лидар во время движения - Когда

нужны нестандартные условия остановки - Чтобы понять, как работает управление!

Упражнение 4.4: Движение на 0.5 метра с циклом

Напиши свой цикл для движения точно на 0.5 метра вперед.

Решение

```
import time
from evabot import Robot, MecanumDrive

robot = Robot()
robot.drive = MecanumDrive(fl=3, fr=4, bl=1, br=2, pattern='X')
robot.start()

time.sleep(1)

start_x = robot.odom.pose.x
robot.drive.move(vx=0.2)

while True:
    distance = robot.odom.pose.x - start_x
    if distance >= 0.5:
        break
    time.sleep(0.01)

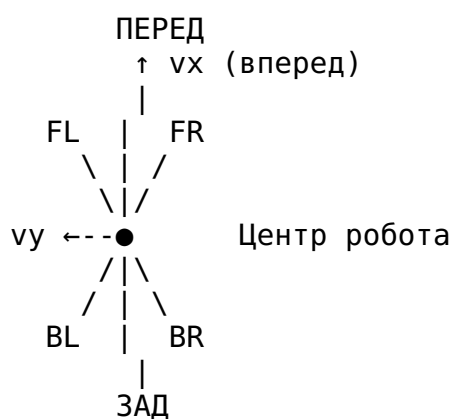
robot.drive.halt()
robot.stop()
```

Система координат лидара

Прежде чем использовать лидар, нужно понять, как он измеряет углы.

Координаты робота vs углы лидара

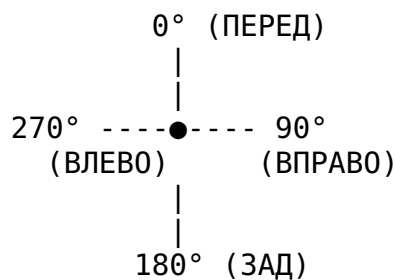
Система координат робота:



• vx = вперед/назад

- vy = влево/вправо
- Робот думает в терминах "вперед", "влево" и т.д.

Система координат лидара:



- Лидар измеряет углы в градусах
- 0° = прямо вперед (то же, что вперед робота)
- 90° = направо
- 180° = назад
- 270° = налево
- Углы увеличиваются **по часовой стрелке** (ЧС): 0° → 90° → 180° → 270° → 360°(=0°)

Главное: Лидар вращается на верхушке робота и измеряет расстояния под разными углами. Когда запрашиваешь угол 0, получаешь расстояние прямо вперед. Угол 90 - направо, и так далее.

Добавление лидара

Лидар вращается и измеряет расстояния до стен и объектов.

Настройка лидара

```
import time
from evabot import Robot, MecanumDrive, RPLidarC1

robot = Robot()
robot.drive = MecanumDrive(fl=3, fr=4, bl=1, br=2, pattern='X')
robot.lidar = RPLidarC1()
robot.start()

time.sleep(3) # Дадим лидару инициализироваться

# Проверяем расстояние впереди
distance = robot.lidar.get_distance_at_angle(0)
if distance:
    print(f"Расстояние впереди: {distance:.2f} м")

robot.stop()
```

Запомни углы: - 0 = перед, 90 = право, 180 = зад, 270 = лево -
 get_distance_at_angle() возвращает расстояние в метрах, или None если нет данных

Остановка перед стеной

Теперь поедem вперед и остановимся перед стеной.

Простое приближение к стене

```
import time
from evabot import Robot, MecanumDrive, RPLidarC1

robot = Robot()
robot.drive = MecanumDrive(fl=3, fr=4, bl=1, br=2, pattern='X')
robot.lidar = RPLidarC1()
robot.start()

time.sleep(3)

# Начинаем движение
robot.drive.move(vx=0.1)

# Продолжаем проверять расстояние
while True:
    distance = robot.lidar.get_distance_at_angle(0)

    if distance and distance < 0.30: # Останавливаемся на 30см
        break

    time.sleep(0.05)

# Стоп
robot.drive.halt()

robot.stop()
```

Как это работает: 1. Начинаем медленное движение вперед (0.1 м/с) 2. Продолжаем проверять расстояние до стены под углом 0 (прямо вперед) 3. Когда расстояние < 0.30 метра (30см), останавливаемся 4. Это предотвращает столкновение!

Упражнение 4.5: Приближение со всех сторон

Заставь робота: 1. Двигаться вперед до 30см от передней стены 2. Двигаться назад до 30см от задней стены 3. Двигаться влево до 30см от левой стены 4. Двигаться вправо до 30см от правой стены

Решение

```
import time
from evabot import Robot, MecanumDrive, RPLidarC1

robot = Robot()
robot.drive = MecanumDrive(fl=3, fr=4, bl=1, br=2, pattern='X')
robot.lidar = RPLidarC1()
```

```

robot.start()

time.sleep(3)

# Вперед
robot.drive.move(vx=0.1)
while True:
    d = robot.lidar.get_distance_at_angle(0)
    if d and d < 0.30:
        break
    time.sleep(0.05)
robot.drive.halt()
time.sleep(1)

# Назад
robot.drive.move(vx=-0.1)
while True:
    d = robot.lidar.get_distance_at_angle(180)
    if d and d < 0.30:
        break
    time.sleep(0.05)
robot.drive.halt()
time.sleep(1)

# Влево
robot.drive.move(vy=0.1)
while True:
    d = robot.lidar.get_distance_at_angle(270)
    if d and d < 0.30:
        break
    time.sleep(0.05)
robot.drive.halt()
time.sleep(1)

# Вправо
robot.drive.move(vy=-0.1)
while True:
    d = robot.lidar.get_distance_at_angle(90)
    if d and d < 0.30:
        break
    time.sleep(0.05)
robot.drive.halt()

robot.stop()

```

Комбинирование движения и сенсоров

Можно комбинировать разные скорости и проверять несколько датчиков.

Движение по диагонали

```
import time
```

```

from evabot import Robot, MecanumDrive

robot = Robot()
robot.drive = MecanumDrive(fl=3, fr=4, bl=1, br=2, pattern='X')
robot.start()

# Движение вперед И влево одновременно
robot.drive.move(vx=0.2, vy=0.2)
time.sleep(2)

robot.drive.halt()
robot.stop()

```

Движение по квадрату

```

from evabot import Robot, MecanumDrive

robot = Robot()
robot.drive = MecanumDrive(fl=3, fr=4, bl=1, br=2, pattern='X')
robot.start()

for i in range(4):
    robot.drive.move_by(dx=0.5)
    robot.drive.move_by(dtheta=1.57)

robot.stop()

```

Упражнение 4.6: Спиральный узор

Заставь робота ехать по растущей спирали: - Начни с 0.2м вперед - Каждая сторона на 0.1м длиннее - Сделай 6 сторон всего

Решение

```

from evabot import Robot, MecanumDrive

robot = Robot()
robot.drive = MecanumDrive(fl=3, fr=4, bl=1, br=2, pattern='X')
robot.start()

distance = 0.2
for i in range(6):
    robot.drive.move_by(dx=distance)
    robot.drive.move_by(dtheta=1.57)
    distance += 0.1

robot.stop()

```

Резюме

Ты изучил:

- Создание меканум-привода с 4 моторами
- Базовые движения: вперед, вбок, поворот с `move()`
- Движение на время с `move_for()` - просто и автоматически
- Чтение одометрии (отслеживание позиции)
- Движение на дистанцию с `move_by()` - точно и автоматически
- Написание своего цикла для нестандартного управления
- Добавление лидара
- Остановка перед стенами используя лидар
- Комбинирование движений для узоров

Три способа движения:

```
# 1. Ручное управление (ты вызываешь halt)
robot.drive.move(vx=0.2)
time.sleep(2)
robot.drive.halt()

# 2. По времени (автоматическая остановка)
robot.drive.move_for(2.0, vx=0.2)

# 3. По дистанции (автоматическая остановка)
robot.drive.move_by(dx=0.5)
```

Шаблон для нестандартного управления с лидаром:

```
# Начинаем движение
robot.drive.move(vx=0.1)

# Проверяем датчик в цикле
while True:
    distance = robot.lidar.get_distance_at_angle(0)
    if distance and distance < 0.30:
        break
    time.sleep(0.05)

# Стоп
robot.drive.halt()
```

Что дальше?

Глава 5 научит тебя более продвинутому управлению с лидаром: - Использование `check_wall()` для выравнивания параллельно стенам - Использование `get_clearance()` для более безопасного обнаружения препятствий - Следование вдоль стен - Навигация по квадратной комнате

Готов? Поехали! □